

Projet QI

Probabilité d'extinction d'espèces dans un système proie-prédateur

Anthony Nouy & Régis Lebrun

2025-2026

1 Description du problème

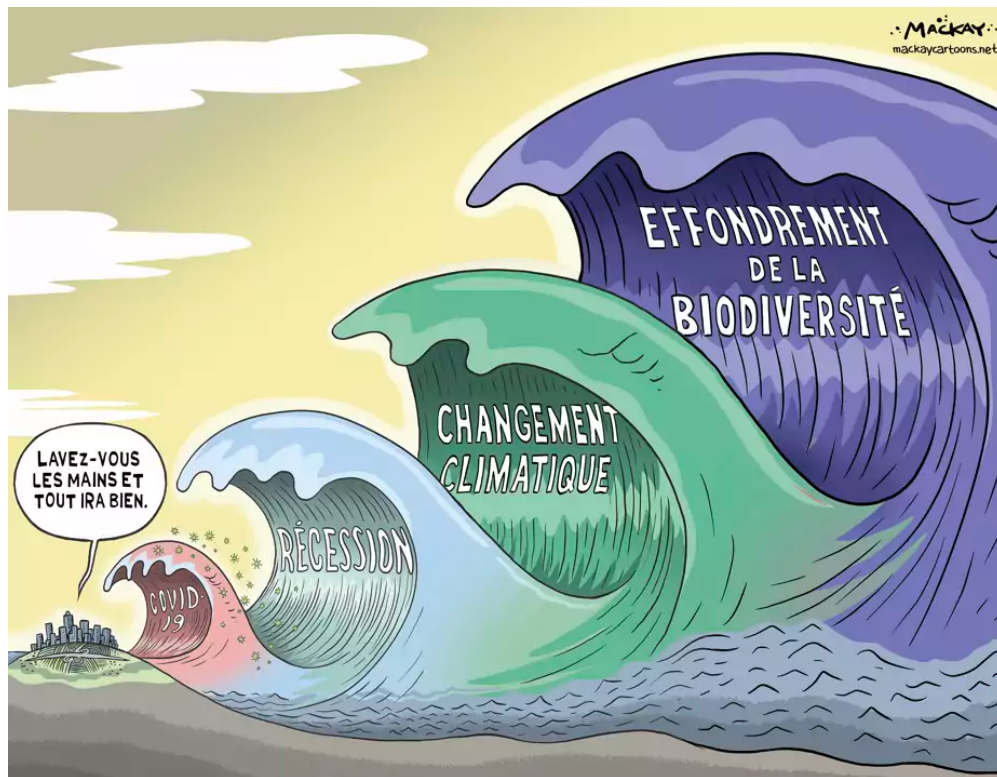


Figure 1: Les crises s'emboîtent!

L'histoire de la Terre est rythmée par de nombreuses crises de la biodiversité parmi lesquelles on distingue cinq crises d'extinction de masse. Une histoire qui résonne avec le déclin actuel de la biodiversité, dont on envisage qu'elle constitue la sixième extinction de masse [Cowie].

La modélisation des interactions entre espèces et avec leur environnement est un outil important dans la compréhension des mécanismes conduisant à de telles disparitions. L'exemple

historique le plus célèbre d'une telle modélisation est le système proie-prédateur de Lotka-Volterra [LotkaVolterra].

Ce modèle simple a été développé historiquement pour modéliser l'évolution des populations de deux espèces en interaction: une population de lièvres dont la source d'alimentation est réputée inépuisable et une population de lynx se nourrissant exclusivement de lièvres.



Figure 2: Lynx poursuivant un lièvre (crédit: Tom & Pat Leeson)

Bien que permettant de reproduire l'évolution cyclique de l'effectif des deux populations conformément aux mesures expérimentales, ce modèle ne permet pas de rendre compte de l'extinction possible de l'une ou des deux populations. Nous allons donc étudier un modèle plus riche, dont l'analyse mathématique est sensiblement plus complexe [Wang], dans lequel la population des proies présente un effet de seuil fort (*Allee effect*):

$$\begin{cases} \frac{dX}{dt} = XQ(X) - c_1\varphi(X)Y \\ \frac{dY}{dt} = -d_2Y + c_2\varphi(X)Y \end{cases} \quad (1)$$

où $Q(X)$ est le taux de croissance par unité des proies, d_2 est le taux de mortalité par prédation, c_1 et c_2 mesurent les interactions entre les proies et les prédateurs (habituellement $c_1 = 1$ et $c_2 < 1$ est l'efficacité de «conversion» des proies en prédateurs) et $\varphi(X)$ est la fonction de réponse des prédateurs, qui correspond à la satisfaction de leur appétit et leur capacité de reproduction.

On impose habituellement les contraintes suivantes:

$$\varphi(0) = 0, \varphi \text{ est croissante, } \varphi(X) \text{ admet une limite } N \text{ quand } X \rightarrow +\infty$$

On s'intéresse à un système présentant un effet de seuil décrit dans [Wang, section 5.1]. Sous

forme adimensionnée, l'évolution de ce système est décrite par le système différentiel:

$$\begin{cases} \frac{du}{dt} = g(u)(f(u) - v) \\ \frac{dv}{dt} = v(g(u) - d) \end{cases} \quad (2)$$

avec $\forall u \geq 0$,

$$\begin{cases} f(u) = \frac{(a+u)(1-u)(u-b)}{bm} \\ g(u) = \frac{mu}{a+u} \end{cases} \quad (3)$$

Les paramètres du modèle sont $a > 0$, $0 < b < 1$, $m > 0$, $d > 0$, auxquels s'ajoutent les conditions initiales $u(0) = u_0 = 1$ et $v(0) = v_0 \in]0, 1[$. Les valeurs nominales de ces paramètres sont données dans le tableau (1).

a_{nom}	b_{nom}	d_{nom}	m_{nom}
0.8	0.1	7.5	14.25

(4)

Table 1: Valeurs nominales des paramètres du modèle proie-prédateur

L'analyse détaillée de ce modèle fait apparaître deux seuils critiques λ et $\bar{\lambda}$ dans l'évolution du système, notamment la position de λ par rapport à $\bar{\lambda}$. Ces seuils sont donnés par les relations

$$\lambda = \frac{ad}{m-d}, \quad \bar{\lambda} = \frac{b+1-a+\sqrt{(b+1-a)^2+3(ab+a-b)}}{3} \quad (5)$$

Pour certaines valeurs de (v_0, a, b, c, d, m) le système tend vers l'état $(u_\infty, v_\infty) = (0, 0)$: il y a extinction complète des deux espèces. Cet état est atteint assez rapidement: on considérera qu'il y a extinction simultanée si $u^2(t_\infty) + v^2(t_\infty) \leq \varepsilon^2$ avec $t_\infty = 20$ et $\varepsilon = 10^{-3}$.

L'objectif de ce projet est de quantifier la probabilité de survenue d'une telle extinction en prenant en compte des incertitudes sur les paramètres du modèle.

2 Travail à réaliser

2.1 Implémentation du modèle

Le modèle nécessite d'intégrer le système différentiel (2) sur l'intervalle $[0, t_\infty]$ pour avoir accès à la valeur de $(u(t_\infty), v(t_\infty))$.

1. Calculer λ_{nom} et $\bar{\lambda}_{\text{nom}}$ à l'aide des relations (5) pour les valeurs nominales des paramètres.

Après lecture de [Wang, section 5], quelles sont les évolutions possibles des trajectoires du système (2) en fonction de v_0 ? On demande une réponse qualitative. Un cycle limite est-il possible?

2. Le système différentiel s'écrit $(u'(t), v'(t)) = F(t, u(t), v(t); a, b, d, m)$ où F est le flot. Créer une fonction symbolique nommée `flot` et prenant en argument, **dans cet ordre**, les variables t, u, v, a, b, d et m .
Évaluer cette fonction aux valeurs nominales et $u = u_0 = 1, v = v_0 = 0.1$ (la valeur de t est indifférente, le système étant autonome). On doit trouver $\text{flot}(0, u_0, v_0, a_{\text{nom}}, b_{\text{nom}}, d_{\text{nom}}, m_{\text{nom}}) = (-0.791667, 0.0416667)$
3. Créer une fonction paramétrique nommée `flotParam` fixant les valeurs des paramètres a, b, d et m à leurs valeurs nominales dans la fonction `flot`.
Évaluer cette fonction en $(0, 1, 0.1)$. On doit trouver $(-0.791667, 0.0416667)$.
4. Créer une fonction paramétrique nommée `Phi` fixant la valeur de t à 0 dans la fonction `flotParam`. C'est cette fonction qui définit le système différentiel à intégrer.
Évaluer cette fonction en $(1, 0.1)$. On doit trouver $(-0.791667, 0.0416667)$.
5. Créer un algorithme d'intégration adaptatif utilisant la méthode de Fehlberg (classe `Fehlberg`) dans son paramétrage par défaut.
Compléter le code suivant pour produire la figure (3):

Listing 1: trajectoires.py

```

1  n_T = 100
   grille = [Tinf * (i / (N_T - 1))**3 for i in range(n_T)]
   # Maillage pour afficher le champ de vecteurs
   maillage = ot.IntervalMesher([12]*2).build(ot.Interval([0.0] * 2, [1.0] *
5     2))
   points = maillage.getVertices()
   champ = ot.Field(maillage, Phi(points))
   ot.ResourceMap.SetAsScalar("Field-ArrowScaling", 1.0)
   g = champ.draw()
   g.setTitle("Trajectoires Lotka-Volterra")
10  g.setXTitle(r'$u$')
   g.setYTitle(r'$v$')
   g.setLegendPosition("topleft")
   nuage = ot.Cloud(points)
   nuage.setColor("black")
15  g.add(nuage)
   # Un ensemble de trajectoires
   palette = ot.Drawable.BuildDefaultPalette(10)
   for i in range(10):
       v_0 = 0.1 + 0.1 * i
20     uv = ...
       trajectoire = ot.Curve(uv)
       trajectoire.setLineWidth(2)
       trajectoire.setColor(palette[i])
       trajectoire.setLegend(rf"$v_0=${v_0}<.1g}")
25     g.add(trajectoire)
   view = otv.View(g, (800, 600), square_axes=True)
   view.save("Figures/ODELotkaVolterra.pdf")
   view.close()
   # Pour afficher la figure dans une cellule de notebook
30  g
   # Pour produire le fichier PDF contenant la figure

```

```
# view=otv.View(g,(800,600),square_axes=True)
# view.save("ODELotkaVolterra.pdf")
# view.close()
```

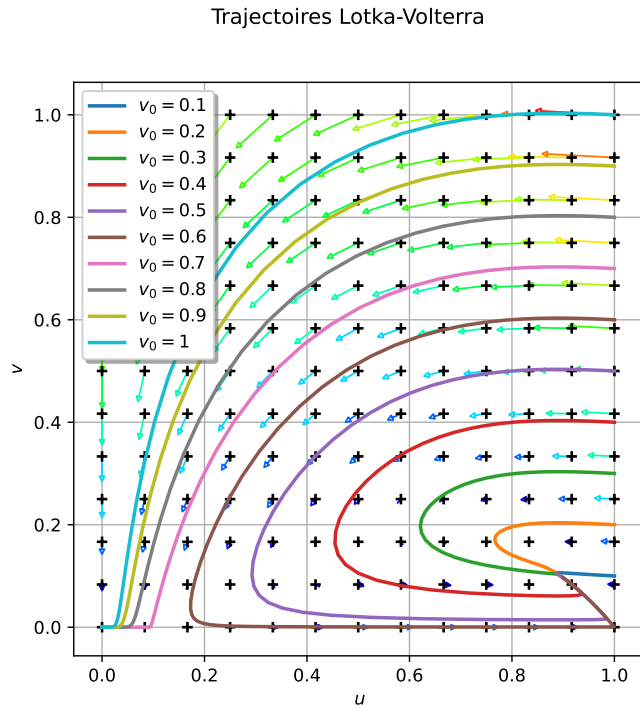


Figure 3: Trajectoires du système Lotka-Volterra pour différentes valeurs de v_0 et autres paramètres à leur valeurs nominales.

Remarque: la grille d'instants en lesquels la solution est calculée n'est **pas** la discrétisation temporelle utilisée par l'algorithme, qui s'adapte automatiquement à la fonction de flot. Cette grille doit **obligatoirement** commencer à 0. On utilise ici une grille non uniforme pour s'adapter à la dynamique du système afin d'avoir des courbes lisses sur la figure.

- Créer une fonction Python nommée `ProiePredateurPy` prenant en argument une liste nommée `parametres` contenant **dans cet ordre** les valeurs (v_0, a, b, d, m) et retournant la liste $[u(t_\infty), v(t_\infty)]$.

On prendra en compte la remarque de la question précédente pour limiter la taille de la structure résultat renvoyée par l'algorithme de résolution.

3 Modélisation probabiliste du système autour de son état nominal

Les paramètres a, b, d, m sont mal connus. Une première modélisation consiste à leur associer des variables aléatoires indépendantes, suivant une loi normale tronquée dont la loi normale sous-jacente de moyenne μ la valeur nominale, d'écart-type σ le dixième de cette valeur nominale, qu'on tronque ensuite à l'intervalle $[\mu - 2\sigma, \mu + 2\sigma]$.

La population initiale de prédateurs est également mal connue. On la suppose uniformément répartie sur l'intervalle $[0.1, 0.5]$.

7. Créer les distributions marginales `aDist`, `bDist`, `dDist`, `mDist` de a , b , d et m . Tracer leurs densités sur une même figure en utilisant une échelle logarithmique en ordonnée. On utilisera la méthode `drawPDF()` des distributions et la classe `Graph`. On regardera en particulier comment changer l'échelle d'un axe et comment modifier la légende d'un tel objet.
Vérifier qu'on a presque sûrement $m > d$.
8. En utilisant l'arithmétique des distributions (dont on justifiera l'utilisation), créer la distribution `lambdaDist` de λ et tracer sa densité. Quelle est sa moyenne? Son écart-type?
Peut-on faire de même pour $\bar{\lambda}$?
9. A-t-on presque sûrement $b > \lambda$? Quelle est la probabilité d'avoir $b > \lambda/4$? On utilisera l'arithmétique des distributions pour réaliser ce calcul.
10. Créer la loi `v0Dist` de v_0 et la loi jointe `parametresDist` de (v_0, a, b, d, m) . Déterminer l'intervalle de confiance bilatéral de niveau 0.99 de cette distribution, ainsi que la probabilité marginale correspondante
(voir la méthode `computeBilateralConfidenceIntervalWithMarginalProbability()`).
11. Créer le vecteur aléatoire `parametresRV` associé à `parametresDist`, la fonction symbolique qui associe à (u, v) le carré de sa norme, puis par composition la fonction `fonctionLimite` qui associe à (v_0, a, b, d, m) la valeur de $u^2(t_\infty) + v^2(t_\infty)$ et enfin créer le vecteur aléatoire `normeCarreeRV` qui est l'image de `parametresRV` par `fonctionLimite`.
12. En fixant la graine du générateur aléatoire à 2026, générer un échantillon de taille 400 de `normeCarreeRV` et tracer sa densité à l'aide d'une reconstruction à noyaux. Doit-on faire une correction de bord? Comment?
En utilisant la loi reconstruite, estimer la probabilité d'extinction. Est-ce une valeur crédible?
13. Créer l'événement `extinction` correspondant à l'extinction des deux espèces. Utiliser l'algorithme `SubsetSampling` pour calculer la probabilité de cet événement. On utilisera les paramètres par défaut et on fixera le coefficient de variation maximum à 0.1. Combien d'itérations ont été nécessaires, pour quel coefficient de variation?
14. Calculer la probabilité de l'événement `extinction` en utilisant l'algorithme `ProbabilitySimulationAlgorithm` et l'échantillonneur `LowDiscrepancyExperiment` en fixant là encore le coefficient de variation maximum à 0.1, et en fixant le nombre d'itérations maximum à 20000 (méthode `setMaximumOuterSampling()`).
Combien d'itérations ont été nécessaires, pour quel coefficient de variation?

4 Modélisation probabiliste du système autour d'un état perturbé

Une perturbation du milieu conduit à une modification des valeurs nominales de b et m . Les nouvelles valeurs sont $b_{\text{nom}} = 0.3$ et $m_{\text{nom}} = 9.5$. On souhaite voir l'effet de cette modification sur la probabilité d'extinction.

15. Modifier les lois `bDist` et `mDist` pour prendre en compte la perturbation du milieu.
A-t-on toujours presque sûrement $m > d$ en supposant ces grandeurs indépendantes?
16. On souhaite modéliser la contrainte $m > d$. Pour cela, créer la loi jointe de (d, m) en utilisant la distribution la plus entropique satisfaisant cette contrainte tout en garantissant les lois marginales imposées à d et m . On utilisera la classe `MaximumEntropyOrderStatisticsDistribution`. Tracer la densité de cette loi.
17. Créer la version mise à jour de `parametresDist` prenant en compte la loi jointe de (d, m) et l'indépendance de cette loi avec la loi des autres paramètres. On utilisera la classe `BlockIndependentDistribution`. Déterminer l'intervalle de confiance bilatéral de niveau 0.99 de cette distribution, ainsi que la probabilité marginale correspondante. Commentaire?
18. Recréer les vecteurs aléatoires `parametresRV` et `normeCarreeRV`.
En fixant la graine du générateur aléatoire à 2026, générer un échantillon de taille 400 de `normeCarreeRV` et tracer sa densité à l'aide d'une reconstruction à noyaux réglée de la même manière que précédemment. On forcera le tracé à se faire sur l'intervalle $[0, 1]$.
En inspectant l'échantillon généré, est-ce qu'une reconstruction à noyaux est raisonnable?
19. Recréer l'événement `extinction`. En fixant de nouveau la graine du générateur aléatoire à 2026, calculer la probabilité d'extinction en utilisant l'algorithme `ProbabilitySimulationAlgorithm` et l'échantillonneur `MonteCarloExperiment` en fixant là encore le coefficient de variation maximum à 0.1, et en fixant le nombre d'itérations maximum à 20000 (méthode `setMaximumOuterSampling()`).
Combien d'itérations ont été nécessaires, pour quel coefficient de variation?

References

- [Cowie] Cowie, R.H., Bouchet, P. and Fontaine, B. «The Sixth Mass Extinction: fact, fiction or speculation?». *Biol Rev*, 97: 640-663 (2022). <https://doi.org/10.1111/brev.12816>
- [LotkaVolterra] Contributeurs de Wikipédia, «Équations de prédation de Lotka-Volterra», Wikipédia, l'encyclopédie libre, https://fr.wikipedia.org/w/index.php?title=https://fr.wikipedia.org/wiki/Atmosph%C3%A8re_normalis%C3%A9e
- [Wang] Wang, J., Shi, J. & Wei, J. «Predator–prey system with strong Allee effect in prey.» *J. Math. Biol.* 62, 291–331 (2011). <https://doi.org/10.1007/s00285-010-0332-1>